

VERIFICATION PLAN

Design and Verification of Parametrized Systolic Array Accelerator

Document Information	
Document ID	[VPL-XXXX-001]
Project / Product	Design and Verification of Parametrized Systolic Array Accelerator
Design Version	[e.g., RTL v1.3 / Netlist v2.0]
Verification Level	Block leve
Prepared By	[Hanisha Dhananjaya, Nikhil Swarna, Venkat Sai Sumanth]
Reviewed By	[Venkatesh Patil, Professor]
Approved By	[Venkatesh Patil, Professor]
Date Created	[2026-04-20]
Last Revised	[2026-04-20]
Revision	[1.0 – Initial Release]
Status	[X] Draft [] In Review [] Approved [] Superseded
Confidentiality	[] Public [X] Internal [] Confidential [] Restricted

Purpose Statement

This Verification Plan defines the strategy, environment, tools, and completion criteria for the functional verification of [Project / Design Name]. It is structured in accordance with the hierarchical verification methodology described in Comprehensive Functional Verification by Bruce Wilde.

FOR ECE-593: FUNDAMENTALS OF PRE-SI VALIDATION: BY: VENKATESH PATIL

Table of Contents

Table of Contents 3

1. Introduction 3

 1.1 Purpose and Scope 4

 1.2 Design Overview 4

 1.3 Verification Hierarchy 5

 1.4 References 5

 1.5 Definitions, Acronyms, and Abbreviations 5

2. Functions to Be Verified 5

3. Functions Not Covered at This Verification Level 6

4. Verification Strategy and Methods 6

 4.1 Overall Strategy 6

 4.2 Testbench Architecture 6

 4.2.1 Testbench Components 7

 4.3 Stimulus Methods 7

 4.3.1 Directed Tests 7

 4.3.2 Constrained-Random Tests **Error! Bookmark not defined.**

 4.3.3 Assertion-Based Verification **Error! Bookmark not defined.**

 4.3.4 Formal Verification **Error! Bookmark not defined.**

 4.4 Checking Strategy **Error! Bookmark not defined.**

5. Verification Environment 8

 5.1 Simulation Environment 8

 5.2 Coverage Plan 8

6. Required Tools 8

7. Completion Criteria 9

 7.1 Simulation Completion Criteria 9

 7.2 Formal Verification Completion Criteria 9

 7.3 Sign-Off Checklist 10

8. Resources and Schedule 10

 8.1 Human Resources 10

 8.2 Hardware Resources 11

 8.3 Schedule 11

9. Risk and Mitigation 11

10. Document Revision History 11

1. Introduction

1.1 Purpose and Scope

This Verification Plan (VP) describes the complete approach for functionally verifying Design and Verification of Parametrized Systolic Array Accelerator. It establishes the verification scope, methods, environment, required tools, completion criteria, and resource plan.

This document applies to the following verification level:

- [X] Block-level verification
- [] Sub-system verification
- [] Full-chip / Top-level verification
- [] System-level verification

1.2 Design Overview

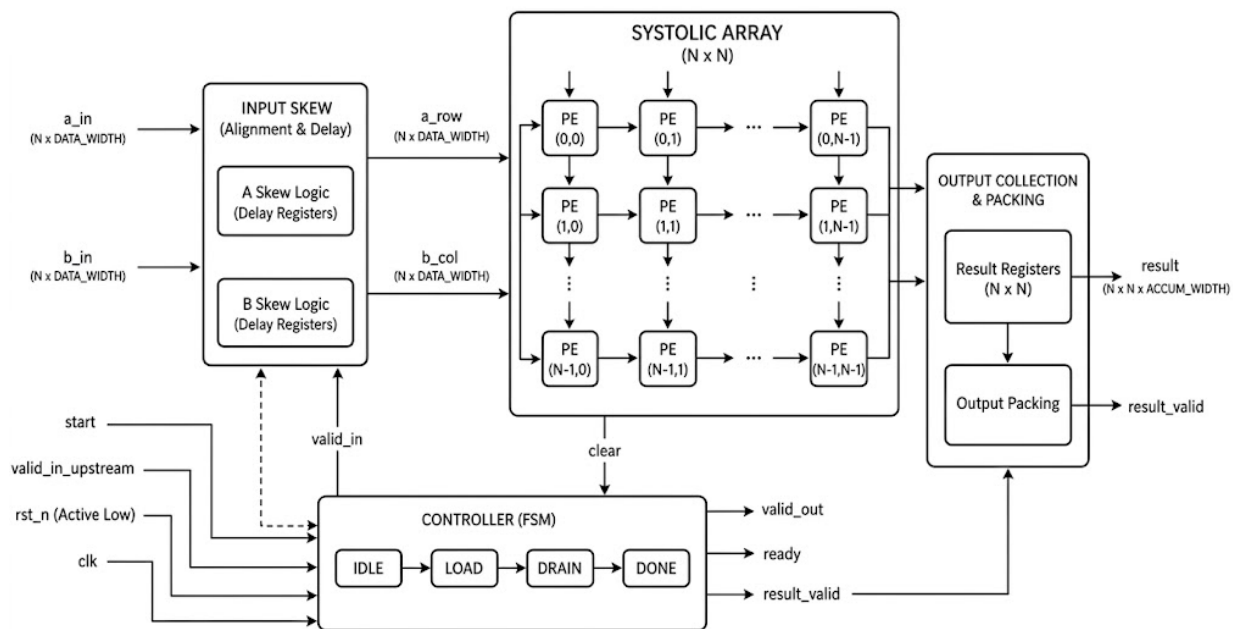
Project Description

The Parametrized Systolic Matrix Multiplication Accelerator is a scalable hardware design implemented in SystemVerilog to efficiently perform matrix multiplication using a parallel systolic array architecture. Matrix multiplication is a fundamental and compute-intensive operation used in applications such as machine learning, signal processing, and scientific computing, where high performance is essential.

The design utilizes an $N \times N$ grid of Processing Elements (PEs), where each PE performs a multiply-accumulate (MAC) operation. Data flows through the array in a structured manner: elements of matrix A propagate horizontally, while elements of matrix B propagate vertically. As data moves through the array, each PE computes partial products and accumulates results over multiple clock cycles, enabling high parallelism and throughput.

To ensure correct timing and alignment of inputs, an input skewing mechanism is implemented using delay registers. A controller based on a finite state machine (FSM) manages different phases of operation, including data loading, computation, and pipeline draining, while generating control signals such as valid, ready, and result_valid.

The accelerator is fully parameterized, allowing configuration of array size and data precision, making it adaptable to various performance requirements. A UVM-based verification environment is used to validate functionality through constrained random testing and comparison with a reference model. Overall, the design demonstrates an efficient and scalable approach to hardware-accelerated matrix multiplication.



1.3 Verification Hierarchy

Verification of this design follows a hierarchical approach consistent with Bruce Wilde's Comprehensive Functional Verification framework. Each level of the hierarchy addresses a specific set of functions and feeds results to the next level.

Hierarchy Level	Design Scope	Covered In This VP	Referenced VP
Block	pe,input_skew,controller,systolic_array	[x] Yes [] No	[VP-ID]
Sub-system	mm_accelerator_top	[x] Yes [] No	[VP-ID]
Full-chip	[Chip name]	[] Yes [] No	[VP-ID]
System	[System name]	[] Yes [] No	[VP-ID]

1.4 References

Ref #	Document Title	Date
[R01]	High-throughput systolic array-based accelerator for hybrid transformer-CNN networks	[2018-12-30]
[R02]	Systolic Array Based Accelerator and Algorithm Mapping for Deep Learning Algorithms	[2024-08]

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
DUV	Design Under Verification
DUT	Device Under Test
UVM	Universal Verification Methodology
SV	SystemVerilog
TB	Testbench
FC	Functional Coverage
CC	Code Coverage
CR	Coverage Report
VP	Verification Plan

2. Functions to Be Verified

This section lists all functions and features of the DUV that are within scope of this verification plan, as required by the verification hierarchy (Wilde, Chapters 3–4). Each function is assigned a unique identifier for traceability.

FV-ID	Function / Feature Description	Design Ref	Priority	Verification Method
FV-001	Reset behavior — all registers (PE accum, skew delays, controller FSM, counters) reset to 0 on active-low rst_n	pe.sv,controller.sv, input_skew.sv	Critical	Directed

Priority legend: Critical = must pass before tape-out; High = required for signoff; Medium = best-effort; Low = stretch goal.

3. Functions Not Covered at This Verification Level

The following functions are explicitly excluded from this plan because they will be verified at a different level of the verification hierarchy. A corresponding Verification Plan reference is provided for each item.

NC-ID	Function / Feature
NC-001	Coverages
NC-002	Assertions
NC-003	Class Based verification
NC-004	UVM Based verification

4. Verification Strategy and Methods

This section defines the overall strategy and the specific test and checking methods that verification engineers will employ, as described in Wilde Chapters 3–4 and 7–12.

4.1 Overall Strategy

The verification strategy is a layered approach that scales across the five course milestones:

- Milestone 1 (Traditional TB): Directed tests with a SystemVerilog reference model — already implemented in tb_mm_accelerator.sv. 4 directed tests running today.
- Milestones 2-3 (Class-based TB): Transition to class-based stimulus generation, transactor/driver/monitor separation, scoreboard class.
- Milestones 4-5 (UVM TB): Full UVM 1.2 environment with agents, sequences, coverage collectors, and constrained-random regression (corner, random, stress, full regression).

4.2 Testbench Architecture

[Describe the high-level testbench architecture. Specify whether UVM is used and at what version. Provide or reference a block diagram of the environment.]

4.2.1 Testbench Components

Component	Description	Reuse / New
tb_mm_accelerator (top)	Instantiates DUV, clock/reset, drives stimulus, checks results	New — implemented (M1)
ref_model() function	Golden SW reference: $C[i][j] = \sum A[i][k] \cdot B[k][j]$	New — implemented (M1)
pack_row / pack_col	Helper functions to pack 2-D A/B matrices into flat N·DATA_WIDTH buses	New — implemented (M1)
unpack_result	Unpacks $N^2 \cdot \text{ACCUM_WIDTH}$ result bus into 2-D array for checking	New — implemented (M1)
run_test task	Parameterized test runner — reset, drive, wait, check, report PASS/FAIL	New — implemented (M1)
UVM Env	Assembles agents, scoreboards, and coverage collectors	New
Agent(s)	One per interface; contains driver, monitor, sequencer	☐ Reused ☐ New
Scoreboard	Compares DUV output against reference model	☐ Reused ☐ New
Reference Model	Golden behavioral model of DUV	☐ Reused ☐ New
Coverage Collector	Implements functional coverage groups and bins	New
Assertion Library	SVA properties bound to DUV interfaces and internals	☐ Reused ☐ New
Register Model (RAL)	UVM register model generated from IP-XACT / RALF	☐ Generated ☐ Manual

4.3 Stimulus Methods

4.3.1 Directed Tests

The four directed tests already implemented in tb_mm_accelerator.sv are:

- TEST 1 - All-Zeros: A=0, B=0, expected result = all zeros. Verifies clear-path and baseline accumulation.
- TEST 2 - Identity × Identity: A=I, B=I, expected result = I. Verifies diagonal data flow and PE indexing.
- TEST 3 - All-Ones: A=[1..1], B=[1..1], expected result element = N. Verifies full MAC path on every PE.
- TEST 4 - Known Random Values: hand-crafted A, B verified against ref_model. Verifies arbitrary arithmetic.

Additional directed tests to be added before Milestone 2:

- Max-operand test A=B=all 0xFF verifies accumulator does not overflow at DATA_WIDTH=8, N=4.
- Reset-during-LOAD assert rst_n mid-transaction, confirm clean recovery on next start.
- Back-to-back - pulse start immediately after result_valid drops; confirm accum cleared by controller.
- valid_in_upstream stall deassert mid-stream, verify no spurious accumulation.

5. Verification Environment

This section describes the simulation and verification environment, in accordance with Wilde Chapters 3, 4, and 7–12.

5.1 Simulation Environment

Simulation Environment Details	
Simulator	Synopsys VCS
Simulator Version	[e.g., VCS 2023.03-SP2]
Verification Methodology	N/A
HDL Language	SystemVerilog IEEE 1800-2017
Assertion Language	N/A
Compile/Sim Options	[Specify key flags: -sv, -uvm, -assert svacomp, etc.]
Seed Management	[e.g., Fixed seed for regression, random seeds for exploration]
Regression Framework	[e.g., Synopsys Verification Compiler Regression / internal Makefile]

5.2 Coverage Plan

[Describe the functional coverage strategy. Define coverage groups and bins for each function listed in Section 2. Reference Wilde Chapter 9.]

FV-ID	Coverage Group Name	Bins / Cross Points	Target (%)	Tool

6. Required Tools

This section lists all software tools required to support the described verification environment, consistent with Wilde Chapters 5–6. Items marked [Procure] may drive the software procurement team; items marked [Internal] require support from internal EDA/flow teams.

Category	Tool / Package	Version	Vendor / Source	Status
Simulator	VCS	[X.X]	Synopsys	[X] Licensed [] Procure
Formal Tool	[VC Formal / JasperGold]	[X.X]	[Synopsys / Cadence]	[] Licensed [] Procure
Coverage Merge	[VCS Coverage / Questa UCB]	[X.X]	[Synopsys / Siemens]	[] Licensed [] Procure

Category	Tool / Package	Version	Vendor / Source	Status
VIP	[Protocol VIP Name]	[X.X]	[Vendor / Internal]	[] Licensed [] Procure
Regression	[LSF / Slurm / Make-based]	[X.X]	[Internal EDA Flow Team]	[] Available [] Internal
Linting	[Spyglass / Questa Lint]	[X.X]	[Synopsys / Siemens]	[] Licensed [] Procure
Clock Domain Check	[Spyglass CDC / 0-In CDC]	[X.X]	[Synopsys / Mentor]	[] Licensed [] Procure
Version Control	[Git / Perforce]	[X.X]	[Internal]	[] Available [] Internal
Bug Tracking	[Jira / Bugzilla]	[X.X]	[Internal]	[] Available [] Internal
Documentation	MS word	[X.X]	[Internal]	[] Available [] Internal

7. Completion Criteria

Verification is declared complete when ALL of the following measurable criteria have been satisfied, in accordance with Wilde Chapter 13. Each criterion must be formally signed off by the Verification Lead and Design Owner.

7.1 Simulation Completion Criteria

#	Criterion	Target Threshold	Achieved Value
1	All directed tests pass with zero unexpected failures	100% pass rate	[] / [%]
2	Constrained-random regression passes across N seeds	$N \geq$ [define] seeds	[] / [N]
3	All functional coverage groups close	$\geq$ [95]%	[] / [%]
4	Statement and branch code coverage achieved	$\geq$ [90]%	[] / [%]
5	Toggle coverage achieved on DUV outputs and interfaces	$\geq$ [85]%	[] / [%]
6	FSM state and transition coverage achieved	100% reachable states	[] / [%]
7	All SVA assertions pass (zero assertion failures in final regression)	0 failures	[] / [N]
8	All open bugs triaged; no Critical or High severity bugs outstanding	0 Critical / High open	[] / [N]
9	Waived coverage items documented and approved	Approval on record	[] Done

7.2 Formal Verification Completion Criteria

#	Criterion	Target	Status
1	All formal properties proven or vacuity-free	PROVEN / BOUNDED	[] Done
2	No unreachable properties (vacuity checks passed)	0 vacuous passes	[] Done
3	All counterexamples analyzed and resolved	0 open CEX	[] Done

7.3 Sign-Off Checklist

Sign-Off Item	Responsible Party	Date	Sign
Simulation completion criteria met (Section 7.1)	Verification Lead	04/19	___
Formal completion criteria met (Section 7.2)	Verification Lead	[Date]	___
Coverage waivers approved	Design Owner	[Date]	___
All high-priority bugs closed or deferred	Design Owner	[Date]	___
Verification plan reviewed and up-to-date	Verification Lead	[Date]	___
Regression results archived	Verification Lead	[Date]	___
Final sign-off approved by Program Management	Program Manager	[Date]	___

8. Resources and Schedule

This section defines the human, hardware, and software resources required, and provides schedule estimates to tie the verification plan to program management, as required by Wilde's framework.

8.1 Human Resources

Role	Name / Team	Allocation (FTE)	Responsibilities
Verification Engineer	Nikhil	33%	Testbench development, test simulation and debugged the design and test bench.
Verification Engineer	Hanisha	33%	Testbench development, Defined methods pack_row, pack_col, unpack_result, ref_model
Verification Engineer	Sai Sumanth	33%	Testbench development, Implemented the functionality of the methods, initialized the signals and matrices.
Design Engineer	Nikhil	33%	Tool setup, Designed the Systolic Array
Design Engineer	Hanisha	33%	Tool setup, Designed the controller

Role	Name / Team	Allocation (FTE)	Responsibilities
Design Engineer	Sai Sumanth	33%	Tool setup, Designed the processing element and input skew.

8.2 Hardware Resources

Resource	Specification	Quantity	Availability
Simulation Compute Servers	[CPU cores, RAM, disk I/O specs]	[N] nodes	[] Available [] Required
LSF / Job Scheduler Capacity	[Max concurrent jobs]	[N] slots	[] Available [] Required
Formal Compute Servers	[RAM-optimized specs for formal]	[N] nodes	[] Available [] Required
Shared File Storage	[NFS/network storage for results]	[X] TB	[] Available [] Required
Emulation Platform	[FPGA-based emulator if needed]	[N] systems	[] N/A [] Required

8.3 Schedule

#	Milestone / Deliverable	Planned Start	Planned End	Owner	Status
M1	Design spec, Verification Plan, RTL, Traditional TB (this document)	04/11	04/19	All	completed
M2	Class-based TB (CLASS_TB), SVA assertions bound	[MM/DD]	[MM/DD]	V-Eng	☐ Open
M3	Class-based TB with scoreboard + basic coverage	[MM/DD]	[MM/DD]	V-Eng	☐ Open
M4	UVM environment — agents, sequences, uvm_env	[MM/DD]	[MM/DD]	V-Eng	☐ Open
M5	UVM regression suite, coverage closure, final sign-off	[MM/DD]	[MM/DD]	V-Lead	☐ Open

9. Risk and Mitigation

#	Risk Description	Likelihood	Impact	Mitigation Plan
R1	RTL instability delaying testbench development	High	High	RTL is already frozen and working (all 4 tests pass). Testbench developed against stable mm_accelerator_top interface with fixed port list

10. Document Revision History

Revision	Date	Author	Description of Change	Approved By
0.1	2026-04-20	[Team — V-Lead]	Initial draft populated from RTL source files (pe.sv, systolic_array.sv, mm_accelerator_top.sv, input_skew.sv, controller.sv, tb_mm_accelerator.sv)	

End of Verification Plan